RK818/RK816 Battery Gauge Developer Guide

ID: RK-KF-YF-18

Release Version: V2.2.0

Release Date: 2021-06-08

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2021. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily introduces the RK818/RK816 sub-module of Rockchip: Battery Gauge. It covers related concepts and functions, DTS configuration, and analysis and positioning of some common issues.

Product Versions

Chip Models	Kernel Version
RK818, RK816	Linux3.10, Linux4.4, Linux4.19

Target Audience

This document (this guide) is mainly intended for the following engineers:

- Technical Support Engineers
- Software Development Engineers

Revision History

Version	Author	Date	Modification Description
V1.0.0	Chen Jianhong	2016-07-25	Initial version
V2.0.0	Chen Jianhong	2017-05-25	Added rk816, rk818 usage in kernel 3.10; Fixed charging voltage/current explanation errors
V2.1.0	Chen Jianhong	2018-05-31	Adjusted document format and layout errors, no main content update
V2.2.0	Chen Jianhong	2021-06-08	Added support for 4.19 kernel

Table of Contents

RK818/RK816 Battery Gauge Developer Guide

- 1. Overview of RK818/RK816 Fuel Gauge
- 2. Principle of Battery Gauge
- 3. Key Concepts
- 4. Drivers and menuconfig
  - 4.1 Battery Gauge Driver Features
  - 4.2 Kernel 3.10
  - 4.3 Kernel 4.4
  - 4.4 Kernel 4.19
- 5. DTS Configuration
  - 5.1 Kernel 3.10
  - 5.2 Kernel 4.4
  - 5.3 Kernel 4.19
  - 5.4 DTS Parameter Description
  - 5.5 Precautions
- 6. Battery Gauge Development
  - 6.1 Preparation
  - 6.2 Issue Resolution
- 7. Battery Calibration
  - 7.1 Principle of Battery Calibration
  - 7.2 Methods for Battery Calibration
  - 7.3 When Calibration is Necessary
- 8. Common Issues Analysis and Troubleshooting

1. Overview of RK818/RK816 Fuel Gauge

RK818/RK816 is a high-performance PMIC that integrates multiple high-current DCDC converters, several LDOs, one linear switch, one USB 5V and boost output, as well as switch charging, intelligent power path management, coulomb counter, RTC, and adjustable power-on timing functions. Among these, "switch charging, intelligent power path management, and fuel gauge (coulomb counter)" are the features covered in this document.

1. Charging Management: It includes functions such as input current limiting, trickle charging, constant current/constant voltage charging, charging termination, and charging timeout safety protection.
2. Intelligent Power Path Management: It can adjust the output voltage to provide the required power to the system load while also charging the battery. When entering the input current limiting state, the input power will be prioritized to supply the system load, and the remaining power will be used for battery charging. Additionally, when the power required by the system load exceeds the limited input power or when the power supply is disconnected, the intelligent power path management function will automatically open the switch between the battery and the system load, allowing the battery to provide additional power to the system load simultaneously.
3. Fuel Gauge (Coulomb Counter): Using its proprietary algorithm, this fuel gauge can accurately measure the battery charge level based on the charging and discharging characteristic curves of different batteries and provide battery charge information to the main system chip through the I2C interface. It also features low-current charging for over-discharged batteries, battery temperature detection, charging safety timer, and chip thermal protection functions.

2. Principle of Battery Gauge

1. Three Basic Principles:

   (1) The open-circuit voltage (OCV) of a battery in relation to the battery's state of charge (SOC) is primarily determined by the materials and manufacturing processes used in the battery, and is minimally affected by temperature and aging, meaning that the SOC-OCV curve remains essentially unchanged after the battery is produced.

   (2) During operation, the battery is influenced by polarization effects, making it difficult to deduce the battery's OCV from the terminal voltage. Therefore, only when the battery is not polarized (such as after a long period of shutdown or during long periods of low current operation) can a usable OCV be obtained, and SOC can be inferred from the OCV.

   (3) A coulomb counter can measure the actual charge flowing into or out of the battery. If the total battery capacity is known, the SOC can be easily determined. However, the cumulative error of the coulomb counter is large, and the total capacity of the battery is affected by factors such as temperature and aging. Therefore, the method using a coulomb counter can only ensure good accuracy over a short period and requires regular updates to the total battery capacity.

   Currently, high-performance battery gauges are based on the real-time remaining capacity status obtained from the above OCV estimation and coulomb counter calculations.

2. Coulomb Counter Method

   A sampling resistor is connected in series at the positive and negative poles of the battery. When current flows through the resistor, a voltage difference (V_delta) is generated. By detecting V_delta, the current flowing through the battery can be calculated. This allows for precise tracking of battery charge changes with an accuracy of up to 1%. Additionally, by combining battery voltage and temperature, the impact of battery aging and other factors on measurement results can be greatly reduced.

3. Current Signal Conditioning

   Using a 20 or 10 milliohm resistor to sample the current, a bias is provided by a constant current source to raise the negative current signal to a positive value. The signal is then amplified by an operational amplifier to the reference voltage Vref range and converted into a digital quantity by the ADC module.

4. Voltage Acquisition

   The battery voltage is divided down (with a division ratio of 0.5) to the Vref range through a voltage divider circuit, and then converted into a digital output by the ADC module.

5. Average Current Acquisition

   The digital part accumulates the corrected current values with a multi-bit accumulator 256 times per second. At the end of one second, the value in the accumulator is divided by 256 to obtain the average current value.

6. Coulomb Counter Update

   The coulomb counter automatically accumulates once during the average current update.

3. Key Concepts

- `Open-Circuit Voltage (OCV)`

  The open-circuit voltage. The PMIC collects this voltage during the power-up sequence because the load at this time is very small, approximately in an open-circuit state, so the voltage at this time is accurate. Purpose: When the power-off time is at least 30 minutes, we consider that the polarization of the battery is basically eliminated, and the obtained OCV voltage is true and effective. Therefore, this voltage is used to query the ocv_table to get a new battery level to update the coulomb counter's value and perform a coulomb counter calibration.

- `OCV Table`

  Each battery has its own battery characteristic curve. Based on the principle that specific voltages correspond to specific battery levels, we divide the battery level from 0% to 100% into 21 points, stepping by 5% battery level, thus obtaining a "voltage<-->battery level" table/group. The purpose of this table is to serve as the basis for the first power-on of the battery, the power-on after a long period of shutdown, and the coulomb counter correction after a long period of sleep. For example:

  ```
  ocv_table = <3400 3599 3671 3701 3728 3746 3762  ......  4088 4132 4183>;
  ```

  Corresponding relationship: 3400mV: 0%, 3599mV: 5%, 3671mV: 10%, ... 4183mV: 100%;

- `Maximum Input Current and Maximum Charging Current`

The maximum current that can be obtained from the adapter is configured in the software, referred to as the "maximum input current." For example, for a 5V/2A adapter, we generally configure the maximum input current in the software as 2A (it can also be set to 1.8A...). RK818/816 has intelligent power path management functionality, which means that the electricity from the adapter is first supplied to the system for use, and the remaining is used for battery charging. The maximum remaining current value allowed to charge the battery, configured in the software, is referred to as the "maximum charging current."

- `Input Current Limitation Occurrence`

  It can be simply understood that when the current demand for supplying power to the battery and the system exceeds the maximum input current, this "insufficient" situation is referred to as "input current limitation has occurred." Alternatively, it can also be understood as: when it is impossible to charge the battery at the maximum charging current under the required conditions of the battery, input current limitation has occurred. This function is mainly used as one of the three conditions for charging termination (the other two are the charging termination voltage and the termination current).

- `Relaxation Mode`

  In extremely low-load situations (currently only for second-level standby), if the system's load current continues to be less than a certain threshold for a certain period of time (configurable in software), the battery gauge module enters the relaxation mode, which can be approximated as an open-circuit state.

- `Relaxation Voltage`

  In relaxation mode, the battery gauge collects a set of voltages every 8 minutes, which is referred to as the relaxation voltage. Purpose: The load in second-level standby is very small, and we approximate that the relaxation voltage is close to the open-circuit voltage. Therefore, in the driver processing, when the system wakes up from second-level standby and meets certain conditions, it uses it to query the ocv_table for coulomb counter correction.

- `Finish Charging Termination Signal`

  When the battery charging termination occurs, the register generates a status signal, referred to as the finish signal. Purpose: The software only considers it to be a true, hardware-based charging termination when it obtains this signal, and then it enters the corresponding algorithm flow to adjust the displayed battery level.

- `Chip Thermal Protection`

  The self-protection mechanism of the PMIC, which is actually a feedback mechanism: when the chip temperature is greater than the set threshold, the process of gradually reducing the input current occurs, thereby reducing the PMIC workload and chip heat. This feedback process is automatically completed by the hardware, and the software cannot participate, nor is there a strict correspondence between temperature and current size. Under extremely harsh conditions, charging may even be stopped.

- `Charging Termination Conditions`

  When the charging current reaches the termination current, the voltage reaches the termination voltage, and there is no input current limitation, a charging termination signal is generated, and charging is no longer continued. We consider that the battery is fully charged at this time. (The condition of input current limitation is needed here to determine whether the charging current has truly decreased or whether the system load is relatively large at this time, resulting in a smaller charging current to the battery)

4. Drivers and menuconfig

4.1 Battery Gauge Driver Features

1. Statistics and display of battery charge level;
2. Setting of charging current and voltage (based on battery and charger types), supporting single-port/double-port charging;
3. 5V power supply for OTG devices;
4. Battery temperature detection.

4.2 Kernel 3.10

rk818 Driver and Macro Configuration:

```
CONFIG_BATTERY_RK818
drivers/power/rk818_battery.c           // Handles battery display + charger detection,
charging voltage, and current settings
```

rk816 Driver and Macro Configuration:

```
CONFIG_BATTERY_RK816
drivers/power/rk816_battery.c           // Handles battery display + charger detection,
charging voltage, and current settings
```

4.3 Kernel 4.4

RK818 Driver and Macro Configuration (Functionality is too vast, hence split into 2 drivers):

```
CONFIG_BATTERY_RK818
CONFIG_CHARGER_RK818
drivers/power/rk818_battery.c           // Handles battery level display
drivers/power/rk818_charger.c           // Handles charger detection, charging voltage, and
current settings
```

RK816 Driver and Macro Configuration:

```
CONFIG_BATTERY_RK816
drivers/power/rk816_battery.c           // Handles battery level display + charger
detection, charging voltage, and current settings
```

4.4 Kernel 4.19

rk818 Driver and Macro Configuration:

```
CONFIG_BATTERY_RK818
CONFIG_CHARGER_RK818
drivers/power/supply/rk818_battery.c    // Handles battery level display
drivers/power/supply/rk818_charger.c    // Handles charger detection, charging voltage, and
current settings
```

rk816 Driver and Macro Configuration:

```
CONFIG_BATTERY_RK816
drivers/power/supply/rk816_battery.c    // Handles battery level display + charger
detection, charging voltage, and current settings
```

5. DTS Configuration

5.1 Kernel 3.10

The node information for RK816 and RK818 is essentially the same, with only a few attributes differing. The following explanation will use RK818 as an example.

A complete battery node information is shown below, which is placed within the RK818 node. The attributes ntc_table, ntc_degree_from, dc_det_gpio, and dc_det_adc are optional parts, while the rest are mandatory.

```
battery {
    compatible = "rk818-battery";  // If it is rk816, change it to "rk816-battery"
    ocv_table = <3400 3599 3671 3701 3728 3746 3762
                 3772 3781 3792 3816 3836 3866 3910
                 3942 3971 4002 4050 4088 4132 4183>;
    ntc_table = <43662 41676 39793 38005 36308 34696 33164
                 31709 30326 29011 27760 26570 25438 24361
                 23335 22358 21427 20540 19695 18890 18121
                 17389 16690 16022 14778 14197 13642 13113
                 12606 12122 11659 11216 10793 10388 10000
                 9629 9273 8933 8607 8295>;
    ntc_degree_from = <1 10>;
    design_capacity = <4000>;
    design_qmax = <4400>;
    bat_res = <120>;
    max_input_current = <2000>;
    max_chrg_current = <1800>;
    max_chrg_voltage = <4200>;
    sleep_enter_current = <300>;
    sleep_exit_current = <300>;
    sleep_filter_current = <100>;   // RK818 does not require this attribute
    power_off_thresd = <3400>;
    zero_algorithm_vol = <3850>;
```

```
    energy_mode = <0>;
    fb_temperature = <105>;
    max_soc_offset = <60>;
    monitor_sec = <5>;
    virtual_power = <0>;
    power_dc2otg = <1>;
    dc_det_gpio = <&gpio0 GPIO_C1 GPIO_ACTIVE_LOW>;
    dc_det_adc = <1>;   // RK818 does not require this attribute, RK816 might need it
};
```

5.2 Kernel 4.4

RK818 DTS Configuration

1. Battery section: Mandatory.

A complete battery node information is shown below, which is placed within the RK818 node. Both the battery and charger sub-device drivers of RK818 will use the information within the battery node. The ntc_table, ntc_degree_from, temperature_chrg_table, and dc_det_gpio are optional parts, while the rest are mandatory.

```
battery {
        compatible = "rk818-battery";
        ocv_table = <3400 3599 3671 3701 3728 3746 3762
                             3772 3781 3792 3816 3836 3866 3910
                             3942 3971 4002 4050 4088 4132 4183>;
        ntc_table = <43662 41676 39793 38005 36308 34696 33164
                              31709 30326 29011 27760 26570 25438 24361
                              23335 22358 21427 20540 19695 18890 18121
                              17389 16690 16022 14778 14197 13642 13113
                              12606 12122 11659 11216 10793 10388 10000
                              9629 9273 8933 8607 8295>;
        ntc_degree_from = <1 10>;
        temperature_chrg_table = <0 10 1000 1>, <0 20 1200 2>, <1 30 1400 5>, <0 40 700 3>;
        design_capacity = <4000>;
        design_qmax = <4400>;
        bat_res = <120>;
        max_input_current = <2000>;
        max_chrg_current = <1800>;
        max_chrg_voltage = <4200>;
        sleep_enter_current = <300>;
        sleep_exit_current = <300>;
        power_off_thresd = <3400>;
        zero_algorithm_vol = <3850>;
        energy_mode = <0>;
        fb_temperature = <105>;
        sample_res = <10>;
        max_soc_offset = <60>;
        monitor_sec = <5>;
        virtual_power = <0>;
        power_dc2otg = <1>;
```

```
        dc_det_gpio = <&gpio0 GPIO_C1 GPIO_ACTIVE_LOW>;
};
```

2. Charger section: Optional.

If Type-C charging is not supported, there is no need for this configuration. For machines that support Type-C charging ports, please add the reference to the “extcon= <&fusbn>” node under the rk818 root node (where n=0,1.., the specific reference should be based on the actual hardware situation), as shown in the figure below. Since rk818_charger.c needs to register the Type-C notification chain based on this reference to obtain the Type-C charger type detection information.

```
rk818: pmic@1c {
        compatible = "rockchip,rk818";
        status = "okay";
        reg = <0x1c>;
        clock-output-names= "xin32k", "wifibt_32kin";
        interrupt-parent= <&gpio1>;
        interrupts = <21 IRQ_TYPE_LEVEL_LOW>;
        pinctrl-names= "default";
        pinctrl-0= <&pmic_int_l>;
        rockchip,system-power-controller;
        rk818,support_dc_chg= <1>;/*1: dc chg; 0:usb chg*/
        wakeup-source;
        extcon = <&fusb0>;              // Very important!
        #clock-cells= <1>;

        battery {
                        ..........
                        ..........
        };
};
```

RK816 DTS Configuration

Please refer to the above RK818 DTS configuration. Difference: RK816 only requires the battery node, no charger-related configuration is needed.

5.3 Kernel 4.19

Please refer to the kernel 4.4 configuration.

5.4 DTS Parameter Description

The following parameter descriptions apply to all kernel versions, but which parameters are supported by each version, please refer to the DTS configurations of the respective kernel versions mentioned above.

- `ocv_table`

Open-circuit voltage-capacity table. That is, "voltage corresponds to capacity," with a total of 21 voltage values, corresponding to 0% -->100%, with a capacity step of 5% between voltage values. This data table can be provided by the battery manufacturer or measured by RK Shenzhen branch, please consult the relevant engineers of the Shenzhen branch.

- `ntc_table`

  Battery NTC table, unit: ohms. If battery temperature detection is required, please fill in the corresponding NTC values. One value represents one temperature, with a temperature step of 1 degree Celsius between adjacent values, increasing from left to right. If battery temperature detection is not required, please remove this property field. The above example indicates the NTC values corresponding to temperatures from -10 to 30 degrees Celsius.

- `ntc_degree_from` and `ntc_degree_from_v2`

  The starting temperature of the ntc_table, that is, the temperature value corresponding to ntc_table[0], unit: degrees Celsius. This property needs to be configured together when configuring the ntc_table.

  (1) Kernels 3.10 and 4.4: Use `ntc_degree_from`

  This property consists of two fields, the first indicates the sign: 1: negative, 0: positive; the second field indicates the temperature magnitude.

  For example: ntc_degree_from = <1, 10> indicates -10 degrees, ntc_degree_from = <0, 10> indicates 10 degrees.

  (2) Kernel 4.19: Use `ntc_degree_from_v2`

  This property consists of one field.

  For example: ntc_degree_from_v2 = <(-10)> indicates -10 degrees, ntc_degree_from_v2 = <10> indicates 10 degrees.

- `temperature_chrg_table` and `temperature_chrg_table_v2`

  Temperature charging table, multiple sets can be filled in. Function: Set the charging current according to the battery temperature range, and default to `max_input_current` and `max_chrg_current` if out of range. If this feature is not needed, please remove this property.

  Reminder: Since this feature was added later, it is recommended that users must check whether the current driver has parsed this property and determine whether it supports this feature.

  (1) Kernel 4.4: Use `temperature_chrg_table`, only supported by RK818.

  Format: `temperature_chrg_table = <parameter1 parameter2 parameter3 parameter4>, ...`

  - Parameter1: Temperature sign, 1: negative, 0: positive
  - Parameter2: Temperature value, used in conjunction with parameter1
  - Parameter3: Charging current
  - Parameter4: Temperature deviation value, when parameter3 is positive, it deviates larger, when negative, it deviates smaller

  Example:

  `temperature_chrg_table = <0 10 1000 2>, <1 30 1400 5>;`

  <0 10 1000 2> means: between [10, 12], set the charging current to 1000mA

  <1 30 1400 5> means: between [-35, -30], set the charging current to 1400mA

  (2) Kernel 4.19: Use `temperature_chrg_table_v2`, supported by both RK818/RK816.

  Format: `temperature_chrg_table_v2 = <parameter1 parameter2 parameter3>, ...`

- Parameter1: Lower temperature limit
- Parameter2: Upper temperature limit
- Parameter3: Charging current

Example:

`temperature_chrg_table_v2 = <(-40) 10 850>, <22 33 1000>, <40 55 1400>;`

<(-40) 10 850> means: between [-40, 10], set the charging current to 850mA

<22 33 1000> means: between [22, 33], set the charging current to 1000mA

<40 55 1400> means: between [40, 55], set the charging current to 1400mA

- `design_capacity`

  Actual battery capacity. The actual available capacity determined after actual measurement. For example, if the nominal capacity is 4000mAh, but the actual measurement is only 3850mAh, then this value should be filled in as 3850.

- `design_qmax`

  Maximum capacity value, mainly used as one of the error correction conditions for software processing. Currently, this value should be filled in as 1.1 times the nominal capacity: that is, nominal capacity * 1.1.

- `bat_res`

  Battery internal resistance, unit: milliohms. Mainly used in the discharge algorithm, very important! This value is obtained at the same time when measuring the ocv_table, so please pay attention to the measurement of this parameter and do not omit it.

- `max_input_current`

  Maximum input current. Currently, there are the following gear positions (unit: mA):

  ```
  RK818: <450, 80, 850, 1000, 1250, 1500, 1750, 2000, 2250, 2500, 2750, 3000>
  RK816: <450, 80, 850, 1000, 1250, 1500, 1750, 2000>
  ```

  Note that the second gear position is 80, not 800! Generally, do not set the 80mA gear position in use.

- `max_chrg_current`

  Maximum charging current. Currently, there are the following gear positions (unit: mA):

  ```
  RK818: <1000, 1200, 1400, 1600, 1800, 2000, 2250, 2400, 2600, 2800, 3000>
  RK816: <1000, 1200, 1400, 1600, 1800, 2000, 2250, 2400>
  ```

- `max_chrg_voltage`

  Maximum charging voltage, that is, the cut-off voltage when the battery is fully charged. Currently, there are the following gear positions (unit: mV):

  ```
  RK818: <4050, 4100, 4150, 4200, 4250, 4300, 4350>
  RK816: <4050, 4100, 4150, 4200, 4250, 4300, 4350>
  ```

- `sleep_enter_current`

  One of the conditions for entering the relaxation mode. Currently, fill in 300, do not change.

- `sleep_exit_current`

One of the conditions for exiting the relaxation mode. Currently, fill in 300, do not change.

- `sleep_filter_current`

  Filter invalid relaxation current. Currently, fill in 100, do not change.

- `power_off_thresd` **Please read and understand carefully**

  The expected system shutdown voltage, unit: mV. Special attention: This value refers to the **instantaneous voltage of VSYS**, not the voltage at the vbat end (but the battery meter collects the voltage at the vbat end)!

  Principle explanation: The voltage at the vbat end needs to pass through a MOS tube with a resistance of about 50 milliohms (in addition, there is also the impedance brought by the PCB wiring) before it is converted into VSYS to supply the system, so using VSYS as the shutdown point basis is correct. From this, we can know: the greater the current load, the lower the voltage at the vsys end; conversely, under the same vsys, the greater the current load, the higher the corresponding vbat voltage.

  The RK platform does not recommend that the voltage at the vsys end be lower than 3.4V, which can easily lead to instability of the VCC_IO output at the rear end of vsys (usually 3.3V). If the hardware design of some platforms has the highest voltage at the rear end of vsys as 3.0V, then the shutdown voltage can be lower than 3.4V.

- `zero_algorithm_vol`

  The voltage value for entering the voltage+coulomb meter discharge mode, unit: mV. Below this value, enter the combined software discharge algorithm of voltage+coulomb meter. Suggestion: for 4.2V batteries, set to 3850mV, and for 4.3V and above batteries, set to 3950mV.

- `energy_mode`

  Some customers may care more about the smoothness of the curve, while others may care more about whether the battery can be completely discharged, and it is difficult to balance the two. Therefore, this property is reserved for selection. When the value is 1, it means to try to discharge the battery as much as possible, and when it is 0, it means to consider the rationality of the curve smoothness as much as possible (for example, setting 3.4V for shutdown, sometimes it may shut down at 3.5V). The driver has tried to balance the smoothness of the curve and the shutdown voltage point as much as possible, so it is recommended to set it to 0. If it is found after testing that the shutdown voltage point really cannot meet the requirements, it can be set to 1 for trial or contact the driver maintainer directly for optimization.

- `fb_temperature`

  The temperature threshold for chip thermal protection, currently there are four temperature gear positions (unit: degrees Celsius):

  ```
  < 85, 95, 105, 115>
  ```

  Currently, VR chooses 115, and the rest choose 105. If set to 0, it means to close the temperature control feedback function, and this value is generally only used for setting when troubleshooting (see "Common Issues Analysis and Troubleshooting"). It is absolutely **not allowed** to close the temperature control feedback function during normal use.

- `sample_res` **Please read and understand carefully**

The resistance value of the sampling resistor at the battery end, unit: milliohms. The coulomb meter knows the current size of the current system through this resistor, please fill in according to the actual size of the resistor pasted on the hardware. Currently, the size of the resistor only supports 20mR (default support) and 10mR (driver support required).

Among them: the coulomb meter drivers on 4.4 and 4.19 kernels of rk818/rk816 both support. The rk818/rk816 on the 3.10 kernel supports the following submissions:

```
fdc31b1 power: rk818-battery: support different ohm sample resistor
0f23c4b power: rk816-battery: support different ohm sample resistor
```

By default, 20mR sampling resistors are generally used. When using 10mR, the range of `max_chrg_current` is doubled:

```
// rk816
10mR: [2000, 2400, 2800, 3200, 3600, 4000, 4500, 4800]
20mR: [1000, 1200, 1400, 1600, 1800, 2000, 2250, 2400]
// rk818
10mR: [2000, 2400, 2800, 3200, 3600, 4000, 4500, 4800, 5200, 5600, 6000]
20mR: [1000, 1200, 1400, 1600, 1800, 2000, 2250, 2400, 2600, 2800, 3000]
```

Instructions:

- After the charging current range is doubled, `max_chrg_current` can still be configured according to actual needs, such as: 4000mA charging is configured as 4000.
- Using a 10mR sampling resistor can double the range of charging current, but the current and capacity (mAh) errors obtained by the coulomb meter are also doubled;
- The doubling of charging current is only a theoretical value, and some values cannot actually be attained. This involves the issue of power conversion. For example, with a 5V/2A charger:

```
5V * 2A = U_bat * I_bat
```

When U_bat takes the minimum 3.4V, I_bat is the maximum value of 2.94A. This can only be achieved without considering energy loss and all current is given to the battery. In fact, the current of the adapter must be allocated to the system first, and the remaining is given to the battery; in addition, the saturation state of the battery itself also needs to be considered.

- `max_soc_offset`

  The maximum capacity error allowed during boot correction. If the system is powered off for at least 30 minutes, an ocv table capacity query will be performed at boot, and compared with the capacity before shutdown. If the deviation exceeds max_soc_offset, a forced correction will be performed, setting the capacity to the true value corresponding to the ocv table. For example: the current displayed capacity is 20%, but the actual capacity calculated according to the ocv voltage is 80%, then the displayed capacity is directly displayed as 80%. This kind of large capacity deviation usually occurs after a system crash, and the size of this value depends on the customer's acceptance, which should be set by the customer themselves. It is not recommended to set this value to less than 60.

- `monitor_sec`

Polling time, unit: seconds. The coulomb meter driver needs to keep polling to work normally, during which a lot of I2C read and write operations are required. However, considering the robustness of I2C on different platforms, this configuration option is reserved. Currently, it is suggested that 5~10s is more appropriate, and setting it to 5s is the best choice.

- `virtual_power`

  Fake battery mode (test mode). Some product forms do not have a battery and are powered by a 5V/2A adapter alone. However, the coulomb meter driver calculates the integral of current and time, not just focusing on voltage. So even though the system input voltage is always 5V when the 5V/2A adapter is plugged in, the displayed capacity will still slowly decrease to 0%.

  Sometimes during the copying process, it is not hoped that the system will shut down or power off due to capacity, charging current, etc. Set this value to 1 to release the charging current limit, and the system input current is always max_input_current to meet the power supply. At this time, the driver always reports to Android that the current is in a charging state and the capacity is 66%.

- `power_dc2otg`

  Whether to support OTG devices to obtain 5V power from DC. For machines that support dual-port charging, the hardware circuit supports the 5V power supply of OTG devices directly from DC when DC is inserted, without the need for RK818 to provide an additional 5V output. Support is set to 1, and if not supported or there is no DC port, please set to 0.

- `dc_det_gpio`

  Specify the GPIO corresponding to the DC pin. If there is no such function, please remove this property.

- `dc_det_adc`

  Whether to support using saradc to detect the DC pin, 1: support, 0: do not support. Generally speaking, this property and "dc_det_gpio" are definitely a choice of two, only rk816 will use it.

5.5 Precautions

- Inability to Completely Disable Battery Charging

  For rk818/rk816, the CHRG_CTRL_REG1[7]: CHRG_EN is used to enable/disable battery charging. However, there is a hardware issue that prevents its normal use. Therefore, developers must not configure it to 0 at any time, as this may lead to stability issues due to insufficient power supply.
  Workaround:

  RK818: Configure the current input current to the 80mA level, to minimize battery charging, but this does not fundamentally solve the problem;

  RK816: Configure the current input current to the 80mA level; or set BAT_CTRL_REG[6]: USB_SYS_EN to 0, which directly disables the input from the adapter side!

- Different Kernel Versions Use Different DTS Configurations: `ntc_degree_from`, `ntc_degree_from_v2`, `temperature_chrg_table`, `temperature_chrg_table_v2`

- Temperature-Charging Current Handling Strategy (only applicable when `temperature_chrg_table_v2` is supported):

  1. If an ntc_table is configured, the maximum input current is limited to 80mA when the battery temperature exceeds the upper limit or falls below the lower limit. Since it is not possible to disable

CHRG_EN to prohibit charging, this method is used to minimize battery charging as much as possible;
2. If a temperature_chrg_table or temperature_chrg_table_v2 is configured, different charging currents are set according to different temperature ranges. If the charging current value is lower than the lowest level supported by the PMIC, it is changed to limit the maximum input current. If it is not within the temperature limit range, it is configured according to max_input_current, max_chrg_current, and adapter type.
3. User requirements for temperature vary, and the driver can be modified accordingly.

6. Battery Gauge Development

6.1 Preparation

1. Measure the OCV curve and internal resistance of the battery: Each battery has a voltage-capacity characteristic curve (ocv_table) and internal resistance, which can be measured by RK Shenzhen Company.
2. Fill in the DTS parameters, please refer to the aforementioned section.
3. Calibrate the battery, please refer to the aforementioned section.
4. Begin normal use.

6.2 Issue Resolution

1. In case of an anomaly during usage, immediately activate debug information to capture the logs from the scene, and then analyze them;
2. If there is a concern about the charging/discharging curve, such as fluctuations or too fast/slow rates, please capture a complete charging/discharging log (0~100% or 100%~0%, principle: try to cover a large battery range);
3. If you cannot determine the cause, capture the logs from the scene or reproduce the issue, and mention it on redmine. Note: Before capturing logs, ensure that the debug information for the energy meter is turned on!! For common issues, please refer to the section "Common Problem Analysis and Localization".

7. Battery Calibration

7.1 Principle of Battery Calibration

1. After the machine is powered off, it is actually only the PMIC that turns off various DCDC/LDO routes, but it does not completely power down. Instead, it maintains a powered state with an extremely low load. The PMIC itself provides some blank data registers that can be used to store data related to the fuel gauge. The information currently stored includes the battery level, coulomb count capacity, and full

charge capacity (i.e., design_capacity) before shutdown. Each time the machine starts up and initializes the fuel gauge driver, these values are not affected by DTS but read the data from before shutdown and continue to use it.
2. When it is necessary to correct the aforementioned three pieces of information, the PMIC must be completely powered down to clear these data. When fully powered down and then powered up again, the battery's GGSTS_REG[4] ("First Power On") status bit will be set. Therefore, it is necessary to remove the battery to achieve this.
3. When the battery is reconnected, the driver determines that it is the "First Power On," and all relevant data will be re-acquired from DTS and calculate the related battery capacity, battery level, etc. In this way, we obtain an accurate state after calibration.
4. For point 3, it is important to note that after removing the battery, it should be confirmed that polarization is essentially eliminated before reconnecting it (the battery can be left to rest until polarization is eliminated). Otherwise, the OCV voltage at startup will also be inaccurate. For example: when discharging with a large load, if the battery is removed, it is actually still in a polarized state, and the battery voltage will slowly rise. If it is reconnected immediately at this time, the collected OCV voltage will be inaccurate!

We must ensure: The battery is in a state where polarization is essentially eliminated during calibration, so that the OCV voltage at the first startup is accurate, and accurate battery level can be obtained.

7.2 Methods for Battery Calibration

There are currently two methods available for calibrating the battery, choose one of the following:

Method One: Hardware Method: Remove the battery for about 10 seconds and then reconnect it.

Method Two: Software Method: Perform the following operations via serial port:

1. Locate the battery node path: `busybox find /sys/ -name bat`, for example, the path might be "/sys/rk818/bat"; (if it's rk816, the path would be "/sys/rk816/bat", same below)
2. Execute: `echo m > /sys/rk818/bat`;
3. Read back to confirm: `echo r > /sys/rk818/bat`, the return value's BIT(4) should be 1 (on rk816, the BIT(1) should be 1).
4. Then shut down the device normally, and wait for at least 30 minutes before turning it on again (this is when you can obtain the accurate OCV voltage).

Additional: If you need to clear the operation in step 2, execute: `echo c > /sys/rk818/bat`;

7.3 When Calibration is Necessary

1. When there is a change in the battery capacity configured in the DTS;
2. When the battery level is noticeably inaccurate (due to reasons such as device crashes, certain special non-battery-capacity stress tests, etc.);
3. Calibrate once before the battery gauge specific burn-in to ensure that the battery is accurate at the start of the test, which is meaningful (only calibrate at the beginning of all test items, no need to calibrate before each case test).

8. Common Issues Analysis and Troubleshooting

1. How to enable debug information and capture logs?

   Method one: Change the `static int dbg_enable = 0` to `1` in the first line of the driver before compiling.

   Method two: If the firmware does not have `dbg_enable` enabled, you can also input the following command via serial port to turn it on or off during runtime:

   Enable: `echo 1 > /sys/module/rk818_battery/parameters/dbg_level`

   Disable: `echo 0 > /sys/module/rk818_battery/parameters/dbg_level`

   If it is rk816, change the node to: `/sys/module/rk816_battery/parameters/dbg_level`

2. Why does the device restart immediately after shutting down with an adapter or USB plugged in, and cannot be powered off?

   The PMIC chip is designed such that it cannot be powered off as long as the charger is plugged in.

3. Why does the status indicated by the GGSTS[4] (battery presence) register not match the actual situation after a hot-swap of the battery after startup?

   The PMIC does not support hot-swap detection of the battery and only performs a detection once during power-on.

4. Why is the voltage after shutdown different from the DTS configured shutdown point, and why does it shut down at such a high voltage?

   The shutdown voltage should be based on the real-time voltage printed in the last log, and this shutdown voltage is the vsys voltage. We need to ensure that the real-time voltage is not lower than the preset shutdown point. After shutdown, the system powers down, and the polarization of the lithium battery slowly disappears, resulting in a voltage recovery process, which is a characteristic of lithium batteries.

5. Why does the battery level not change much during power on/off, reboot, and second-level standby, but the voltage slowly decreases?

   At this time, the driver cannot poll and enter the working state normally, so it does not support this type of stress test. The quality department and customers do not need to test these 3 items when performing battery meter tests.

6. Why is the charging current always small when using a battery charger?

   (a) Confirm whether the charging cable is of poor quality and high impedance, resulting in the actual voltage supplied to vbus being far less than 5v. You can externally connect a regulated power supply and appropriately increase the voltage to observe if the current can increase;

   (b) Enter the screen-off state (first-level standby) and observe if the charging current increases to determine if it is related to the operating power consumption;

   (c) A nearly fully charged battery will definitely have a small charging current, so please pay attention to the battery voltage;

   (d) In high-temperature and high-load situations, it may be that the PMIC temperature has risen and triggered input current limiting. So first increase the feedback temperature (fb_temperature) to observe if it is effective. If not, directly disable the temperature control by setting fb_temperature to 0.

7. What happens to the battery level after removing the battery and then powering on? Our requirement is that the battery level does not change after removing the battery. Can this be met?

   (a) After removing the battery, the PMIC completely loses power, and at this point, powering on can only query the ocv voltage and retro-calculate the battery level from the ocv_table, so it is normal, representing a re-calibration of the battery;

   (b) Hope that the battery level does not change after removal? It is almost impossible, unless the software avoids it: write the battery level before shutdown to a file, and then read it after power-on. If the customer has a requirement, please ask the customer to add this part of the avoidance processing code themselves.

8. The PMIC has a temperature feedback function to regulate the input current. So, how can we know the internal temperature of the PMIC at this time?

   There is no way to know, as this function was not included in the design.

9. Why is the current printed on the log so outrageous, with reversed positive and negative signs or the current size being so different from the actual value?

   Please confirm that a 20/10 milliohm sampling resistor is used and that the resistor has high precision; secondly, please confirm that the solder joints are clean, and the sampling resistor should be located between BAT- and GND.

10. Why is the finish reported when the voltage is not full, such as only 3.9V?

    Generally, it is because the battery quality is not good, and the maximum charging current set is too large, causing the battery board to trigger self-protection, resulting in the PMIC misreporting the finish state.

11. Why does the current value in the finish state fluctuate between positive and negative (with a small value)?

    This is the zero-point error of the current after full charge, and it is not a problem.

12. Why is the reported finish with only 90% or so of the battery level displayed, not 100%?

    Due to the cumulative error of the coulomb counter, and the charging end is completed by hardware, there will be a certain error between the two, and it is not possible to grasp the timing of the finish report so accurately, so this situation is normal. In this case, the software will do a processing, that is, slowly let the battery level approach 100%, which is imperceptible to the end user, and there is no problem in use.

13. Why is the battery level so inaccurate during operation, and why is it so different from the values in the ocv_table?

    Concept confusion. The ocv_table is the ratio of `voltage-battery level` in the open-circuit state without load, and we only use this table during startup calibration and sleep correction. So such a comparison is meaningless, and it does not make sense in principle.

14. Is it possible to replace batteries with different specifications?

    Not supported. If the battery is changed, then the ocv curve, internal resistance, capacity, and other parameters need to be retested and filled in.

15. What are the conditions for the PMIC to determine that the battery is full?

    Three conditions must be met simultaneously: the voltage reaches the cutoff voltage, the current reaches the finish current, and there is no input current limiting.

16. Why does the battery icon always show 50% charging?

Please disable the test_power driver.

17. Why does the battery icon always show 66% charging?

    There is currently no battery connected; or the DTS's virtual_power is configured as 1, please configure it to 0.

18. Why is the charger insertion/removal not recognized?

    (a) Kernel 4.4

    rk818_charger.c is responsible for charger detection, and the insertion/removal of chargers and otg devices on the USB port both depend on the USB notification chain. Please pay attention to whether the serial log prints “rk818-charger: receive xxx notifier event: xxx” and other serial information. If not, it may be that the USB notification chain has not been registered successfully (unlikely), or there is a problem with the USB driver.

    (b) Kernel 3.10

    rk818_battery.c is responsible for charger detection (including battery level), and the insertion/removal of chargers and otg devices on the USB port both depend on the USB notification chain. Please pay attention to whether the serial Log prints “rk818-bat: receive xxx notifier event: xxx” and other serial information. If not, it may be that the USB notification chain has not been registered successfully (unlikely), or there is a problem with the USB driver.